

Ejercicio1

```
[2]: miLength :: [a] -> Int
miLength [] = 0
miLength (x:xs) = 1 + miLength (xs)
miLength [1,4,6,9,11]
```

Use foldr

Found:

```
miLength [] = 0
```

```
miLength (x : xs) = 1 + miLength (xs)
```

Why Not:

```
miLength xs = foldr (\ x -> (+) 1) 0 xs
```

Redundant bracket

Found:Why Not:

```
(xs) xs
```

```
5
```

```
[10]: miLast :: [a] -> a
miLast (x:xs) = x
miLast (x:xs) = miLast xs
miLast [2,4,5,6,7,8]
```

```
2
```

```
[16]: duplicar :: [Int] -> [Int]
duplicar [] = []
duplicar (x:xs) = x*2 : duplicar xs
duplicar [3,9]
```

Use map

Found:

```
duplicar [] = []
```

```
duplicar (x : xs) = x * 2 : duplicar xs
```

```
[6,18]
```

Why Not:

```
duplicar xs = map (* 2) xs
```

```
[18]: eliminarPares :: [Int] -> [Int]
eliminarPares [] = []
eliminarPares (x:xs)
  | even x = eliminarPares xs
  | otherwise = x : eliminarPares xs
eliminarPares [3,6,8,7,9,11]
```

```
[3,7,9,11]
```

```
26]: duplicar xs = map (* 2) xs
      duplicar [2,4,6]
```

Eta reduce

Found:

```
duplicar xs = map (* 2) xs
duplicar [2,4,6]
[4,8,12]
```

Why Not:

```
28]: isImpar x = x `mod` 2 == 1
      filter isImpar [1, 2, 3, 4, 5, 6]
[1,3,5]
```

```
[*]: miLast :: [a] -> a
      miLast = foldl1 (\_ a-> a)
      print(miLength [3,5,7,8,9,2,3])
```

Ejercicio 2

```
5]: divisoresTres x = x `mod` 3 == 0
      filter divisoresTres [1,2..30]
[3,6,9,12,15,18,21,24,27,30]
```



```
0]: [ x `mod` 3 == 0 | x <- [1,2..30], even x]
[False,False,True,False,False,True,False,False,True,False,False,True,False,False,True]
```

```
]:
```